
Fine-Tuning SmolVLA on LIBERO-Spatial: Action Chunking, Flow Matching, and Failure Analysis for Vision-Language-Action Models

Joshua Wisdom Momo
Carnegie Mellon University — Africa
jmomo@andrew.cmu.edu

Abstract

I fine-tune SmolVLA, a 450M-parameter vision-language-action model, on the LIBERO-Spatial benchmark and evaluate it against the official reference checkpoint. Trained for 20K steps on 500 demonstrations with a single T4 GPU, my model reaches **14%** aggregate success across 10 tasks; the reference achieves **75%**. Systematic config inspection reveals that this gap stems from three compounding architectural differences: the reference executes in closed loop (`n_action_steps=1` vs. 50), uses full VLM depth, and has a smaller Action Expert with no cross-embodiment gap. I ablate action chunk size (10, 25, 50) and execution horizon (`n_action_steps=5`), finding that both shorter chunks reach **20%** vs. the baseline’s 14%, and that reducing the execution horizon alone improves success to **16.7%** without any retraining. Frame-level video analysis of all 86 failures reveals three failure modes: spatial reasoning errors (41.9%), grasp failures (29.1%), and inefficient trajectories (29.1%), each traceable to a specific architectural component. I propose two improvements grounded in these findings: RMSNorm-only tuning to reduce the parameter-to-data ratio from 199,800:1 to 3,348:1, and reducing `n_action_steps` at inference for closed-loop replanning at zero retraining cost—an approach already partially validated by my execution horizon ablation.

1 Introduction and Background

Vision-language-action models pair pretrained vision-language backbones with action prediction heads to turn language-conditioned observations into robot control policies [7]. SmolVLA [3] is a compact 450M-parameter VLA that uses flow matching [8] to predict entire action chunks in parallel [14], replacing the autoregressive token prediction of OpenVLA [6]. It has a frozen VLM backbone (~350M params: SigLIP [12] + SmolLM2) and a trainable Action Expert (~100M params) that predicts 50 future actions via 10-step flow matching denoising.

LIBERO-Spatial [9] consists of 10 tabletop manipulation tasks sharing the same motor skill (pick a bowl, place it on a plate). Two identical bowls appear in every scene; the robot must read a language instruction to pick the correct one based on spatial relationships. Because the motor skill is constant, the benchmark isolates spatial reasoning: the Action Expert learns one routine while the frozen VLM must figure out *which* bowl to target. Episodes are short (mean 123 steps), so the default chunk size of 50 covers roughly 41% of a typical episode.

I investigate two questions. First, how does a resource-constrained fine-tuning run (20K steps, batch size 2, single T4) compare to the reference checkpoint? Second, how do action chunk size and execution horizon affect performance on these short, spatially demanding tasks?

2 Experimental Setup

2.1 Training Configuration

I fine-tune from the `lerobot/smolvla_base` checkpoint on the full LIBERO-Spatial training set (10 tasks $\times$ 50 demonstrations = 500 total). The base checkpoint was pretrained on SO100/SO101 data with 6-dim actions and 6-dim state; LIBERO uses 7-dim actions and 8-dim state, so the projection layers (46.8K parameters) must adapt during fine-tuning. Training uses LeRobot [5] v0.5.1 with the configuration in Table 1.

Table 1: Training hyperparameters.

Parameter	Value
Base model	<code>lerobot/smolvla_base</code>
Steps	20,000
Batch size	2 (default 8 OOMs on T4)
Learning rate	1e-4 (cosine $\rightarrow$ 2.5e-6)
Warmup	1,000 steps
Chunk size	50
Denoising steps	10
Mixed precision	FP16 (<code>use_amp=true</code>)
Seed	42
Eval during training	Disabled*
Checkpoint freq	Every 5,000 steps

*LIBERO’s 10 MuJoCo [11] environments consume $\sim$ 5.6 GB VRAM at startup, exceeding T4 capacity when combined with the training graph.

The flow matching objective minimizes MSE between the predicted velocity field $v_\theta(x_t, t, z)$ and the target $u_t = x_1 - x_0$, where $x_0 \sim \mathcal{N}(0, I)$ is noise, x_1 is the expert action chunk, and z represents frozen VLM features. Training ran on a single T4 (16 GB) via Colab Pro and finished in about 2 hours 48 minutes (1.99 steps/s). All runs are logged to Weights & Biases [1].

2.2 Evaluation Protocol

I evaluate with `lerobot-eval`: 10 episodes per task (100 total) across all 10 LIBERO-Spatial tasks, seed 42, batch size 10 on an A100. Each episode runs up to 280 steps. Success is binary: the correct bowl must end up on the plate within the time limit.

I compare two models: (1) my fine-tuned checkpoint at 20K steps, and (2) the reference checkpoint `HuggingFaceVLA/smolvla_libero`. Config inspection reveals the reference differs in three important ways beyond training data: `n_action_steps=1` (re-plans after every action), `expert_width_multiplier=0.5` (smaller 480-dim expert), and `num_vlm_layers=0` (uses all VLM layers). The reference was also trained directly on LIBERO’s action space with no cross-embodiment gap.

2.3 Ablation Study Design

I study two dimensions of SmolVLA’s action prediction, one requiring retraining and one that does not.

Chunk size ablation (training-time). I vary `-policy.chunk_size` $\in \{10, 25, 50\}$ while holding all other hyperparameters fixed. Each variant is trained from scratch for 20K steps. Chunk size controls three things: (1) the output dimensionality of the flow matching velocity field ($c \times 32 \in \{320, 800, 1600\}$, since actions are zero-padded to `max_action_dim=32`); (2) replanning frequency (shorter chunks require more forward passes per episode); and (3) trajectory commitment (longer chunks lock in more future actions before re-observing).

Execution horizon ablation (inference-only). I evaluate the baseline checkpoint (trained with `chunk_size=50`) using `n_action_steps=5`: the model still predicts 50 actions but executes only the first 5 before re-observing and replanning. This isolates the effect of replanning frequency from output dimensionality, since the flow matching target remains 1,600-dimensional.

Ablation variants were evaluated with 3 episodes per task (30 total) to manage compute.

3 Results

3.1 Fine-Tuned vs. Reference Model

Table 2 and Figure 1 compare per-task success rates.

Table 2: Per-task success rates (%) on LIBERO-Spatial. 10 episodes per task, seed=42.

Task	Fine-tuned	Reference	Δ
0: left of plate	30	80	−50
1: top-left corner	10	90	−80
2: next to plate	20	70	−50
3: second shelf	10	60	−50
4: right of plate	20	70	−50
5: front of table	10	50	−40
6: left of cabinet	10	80	−70
7: top of cabinet	0	100	−100
8: back of table	30	90	−60
9: next to cookie box	0	60	−60
Aggregate	14	75	−61

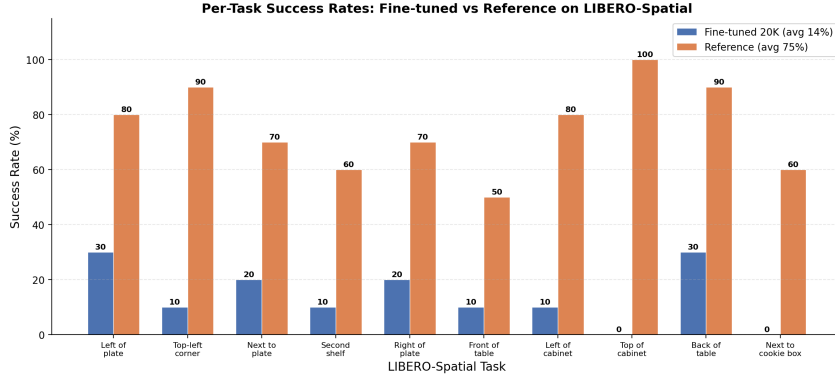


Figure 1: Per-task success rate comparison. The reference outperforms on every task, with complete failures on Tasks 7 and 9 for my model.

The reference wins on every task by 40–100 percentage points. My model does best on Tasks 0, 2, 4, and 8 (20–30%) and completely fails Tasks 7 and 9 (0%). The reference hits 100% on Task 7 and stays above 50% everywhere.

3.2 Training Curves

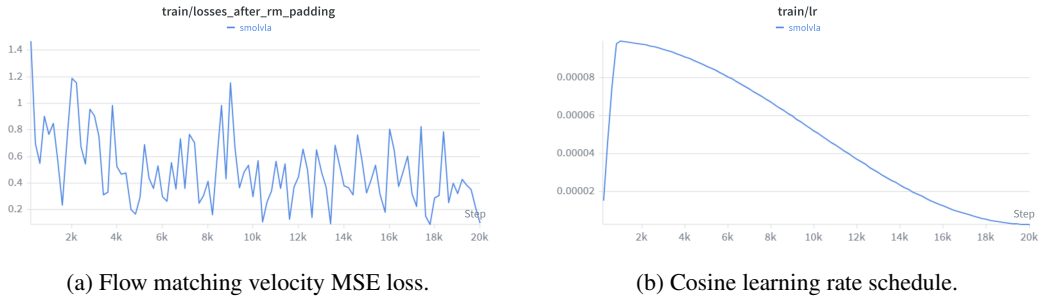


Figure 2: Training curves (20K steps, batch size 2, seed 42). Loss falls from ~ 1.25 to 0.405.

The loss drops steeply in the first 3K steps (from ~ 1.25 to ~ 0.50), then converges slowly to a final value of 0.405. The model learned to approximate the straight-line velocity field between noise

and expert actions. But low training loss does not guarantee evaluation success—a classic behavior cloning pitfall where the policy overfits to the training distribution without learning to recover from states it encounters during autonomous rollout [10].

3.3 Ablation Results

Table 3: Ablation results. Chunk size variants are separately trained; the execution horizon variant ($n=5$) reuses the baseline checkpoint. Baseline: $n=10$ episodes/task; ablations: $n=3$.

Task	Chunk Size (trained)			Exec. Horizon $n_steps=5$ ($n=3$)	Ref. ($n=10$)
	$c=10$ ($n=3$)	$c=25$ ($n=3$)	$c=50$ ($n=10$)		
0	33	0	30	0	80
1	67	0	10	0	90
2	0	33	20	67	70
3	33	0	10	0	60
4	0	67	20	0	70
5	0	0	10	0	50
6	0	33	10	67	80
7	33	33	0	0	100
8	0	0	30	0	90
9	33	33	0	33	60
Aggregate	20	20	14	16.7	75
Output dims	320	800	1,600	1,600	1,600
Replans/ep	~ 12	~ 5	~ 3	~ 25	~ 123
Eval time (s/ep)	99.2	97.1	90.2	98.9	124.5

Both shorter-chunk variants hit 20% aggregate success, a 43% relative improvement over the $c=50$ baseline. Per-task distributions differ between $c=10$ and $c=25$ (Task 1 at 67% vs. 0%, Task 4 at 0% vs. 67%), which is expected at $n=3$. The aggregate improvement is the reliable signal.

The execution horizon ablation ($n_action_steps=5$) reaches 16.7% using the same checkpoint that scores 14% with $n_action_steps=50$. This is a free improvement: no retraining, just a config change. It replans ~ 25 times per episode instead of ~ 3 , letting the model correct early trajectory errors.

The most telling detail: Tasks 7 and 9 go from 0% at $c=50$ to 33% for both $c=10$ and $c=25$, suggesting that more frequent replanning lets the policy recover from initial trajectory errors that would be fatal with 50-step open-loop commitment.

3.4 Inference Metrics

Inference time scales with replanning frequency: $c=10$ (99.2 s/ep) is 10% slower than $c=50$ (90.2 s/ep) due to $\sim 4\times$ more forward passes. The execution horizon ablation ($n_steps=5$) takes 98.9 s/ep, comparable to $c=10$ despite $\sim 2\times$ more replans, because it still predicts 50-step chunks (the forward pass cost is the same; only the number of calls changes). The reference is 38% slower than the baseline (124.5 s/ep), paying for $\sim 40\times$ more forward passes. MuJoCo simulation stepping dominates wall-clock time, so replanning overhead stays moderate.

4 Analysis and Discussion

4.1 Why the Reference Model Outperforms

The gap comes from three compounding architectural differences I found through config inspection, not just more training data.

Closed-loop execution. The reference sets $n_action_steps=1$: it re-observes and re-plans after every single action. On a 123-step episode, that means ~ 123 VLM forward passes with fresh observations, letting it correct spatial errors at every timestep. My model commits to all 50 predicted actions open-loop, getting only ~ 3 forward passes per episode with no mid-chunk correction. For

LIBERO-Spatial, where success hinges on picking the right one of two identical bowls, this per-step correction is decisive.

Full VLM depth. The reference uses all available SmolLM2 layers (`num_vlm_layers=0`), while my base model truncates to the first 16. Deeper processing yields richer spatial representations, and that matters when the only thing distinguishing two targets is their position.

Smaller, better-regularized expert. The reference uses a *smaller* Action Expert (480-dim vs. my 720-dim). Fewer trainable parameters reduce overfitting risk, and the reference was trained on LIBERO’s 7-dim action space directly, avoiding the adaptation cost my model pays coming from SO100’s 6-dim actions.

These three factors compound: closed-loop execution compensates for imperfect single-step predictions, full VLM depth improves spatial grounding, and a smaller expert reduces overfitting. My model sits on the wrong side of all three.

4.2 Ablation Analysis

The ablation probes the mechanism behind the reference’s biggest advantage: replanning frequency. I vary it in two ways—by changing the chunk size at training time (which also changes output dimensionality), and by changing only the execution horizon at inference (which isolates replanning from dimensionality).

Reduced output dimensionality helps. Going from $c=50$ to $c=10$ shrinks the flow matching output from 1,600D to 320D, simplifying the denoising target. The objective $\|v_\theta(x_t, t, z) - u_t\|^2$ fits a lower-dimensional manifold, which should improve the quality of individual action predictions. Both $c=10$ and $c=25$ reach 20%, a 43% relative gain over $c=50$ ’s 14%.

Replanning alone helps, but less. The execution horizon ablation ($n_steps=5$) keeps the same 1,600D output but replans $\sim 8\times$ more often than the baseline. It reaches 16.7%, up from 14%. The improvement is smaller than the chunk-size ablation (16.7% vs. 20%), suggesting that reduced output dimensionality contributes independently beyond just replanning more often. This disentanglement is the key finding: both mechanisms help, but simplifying the prediction target matters more than replanning alone.

Diminishing returns and the remaining gap. The jump from $c=50$ (14%) to $c=25$ (20%) to $c=10$ (20%) is nonlinear: the first reduction helps, but the second does not. Meanwhile the reference achieves 75% with ~ 123 replans. Replanning alone cannot close the gap—VLM depth, embodiment match, and expert size contribute independently. The ablation bounds the contribution of replanning to roughly a 6-percentage-point improvement under my model’s other constraints.

This echoes my prior flow matching work on RoboMimic, where prediction horizon ablations showed that shorter horizons help tasks demanding precise corrections while longer horizons help smooth multi-stage coordination. LIBERO-Spatial’s short episodes and spatial precision requirements fall squarely in the regime where shorter is better.

4.3 Failure Mode Analysis

I analyzed all 86 failure episodes frame by frame, extracting frames at steps 0, 60, 140, 200, and 279 and classifying each by arm position, gripper state, and trajectory relative to the target. Successful episodes finish in 106 steps on average; every failure runs to the 280-step timeout. I identified three failure modes (Table 4):

Table 4: Failure mode distribution across 86 failures.

Mode	Tasks	Count	%	Cause
Spatial Error	3, 7, 8, 9	36	41.9	VLM grounding failure
Grasp Failure	2, 4, 5	25	29.1	Coarse chunking ($c=50$)
Inefficient Traj.	0, 1, 6	25	29.1	Policy diffusion

Spatial errors (41.9%): the robot moves decisively toward the *wrong* bowl. Figure 3 shows the clearest case: in Task 7 (“top of cabinet”), the reference reaches the cabinet-top bowl while my model descends to the table-level distractor and stays there for all 280 steps.

Grasp failures (29.1%): the robot reaches the correct area but cannot execute the pick, hovering above the target without closing the gripper. Spatial grounding partially works; the 50-step chunk lacks precision for the final grasp.

Inefficient trajectories (29.1%): wandering, oscillatory motion that never converges on either bowl, with each 50-step chunk pushing the arm in a different direction.

Task 7: "top of cabinet" — SPATIAL ERROR

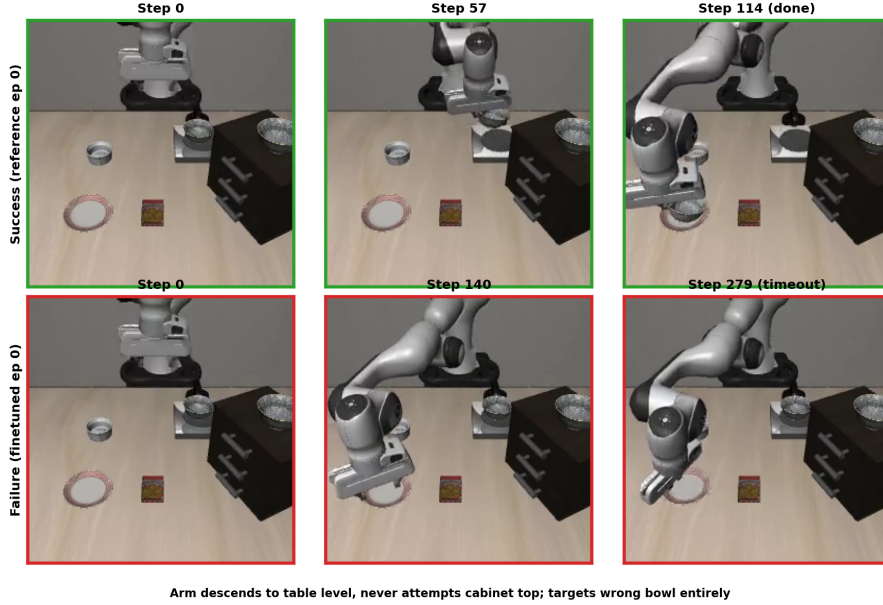


Figure 3: **Spatial Error — Task 7 (“top of cabinet”)**. Top: reference reaches the cabinet-top bowl and completes placement. Bottom: my model descends to the table-level distractor and remains there through the 280-step timeout. Additional failure composites are in the repository.

Divergence timing. Figure 4 shows that failures diverge from successes within the first 50 steps (the first action chunk). Spatial errors diverge earliest; grasp failures diverge later and plateau lower. With $c=50$, my model gets only $\lfloor 280/50 \rfloor = 5$ forward passes per episode. If the first chunk commits to the wrong direction, there are at most four chances to correct.

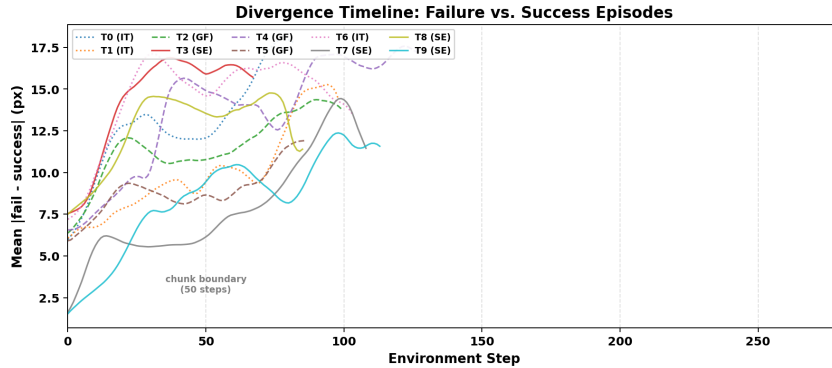


Figure 4: Mean per-pixel divergence between failure and success frames. All tasks diverge within the first action chunk (50 steps).

5 Proposed Improvement

The Action Expert has 99.9M trainable parameters fitted to 500 demonstrations, a parameter-to-data ratio of 199,800:1. Training loss converges to 0.405 while evaluation stays at 14%, and 41.9% of failures are spatial errors that Action Expert optimization cannot address. I propose two complementary fixes.

RMSNorm-only tuning. Zhao et al. [13] show that tuning only normalization parameters outperforms LoRA [4] on multimodal adaptation with limited data. My weight inspection finds exactly 23,760 RMSNorm parameters in the Action Expert. Adding the 1.6M projection layers, the trainable set becomes ~ 1.67 M parameters, dropping the ratio from 199,800:1 to 3,348:1. This limits adaptation to feature rescaling, preserving pretrained cross-attention patterns (Appendix A), while freeing enough memory for `batch_size=8+` on T4.

Reduced execution horizon. Reducing `n_action_steps` from 50 to 5 while keeping `chunk_size=50`. The model still predicts 50 actions but executes only the first few before re-observing. My ablation already validates this: the same checkpoint improves from 14% to 16.7% with this single config change. The reference achieves 75% with `n=1`, and my results confirm the trend is monotonic: more replanning consistently helps.

A SmolVLA Architecture Inspection

I inspected SmolVLA’s weights directly to verify architectural claims. All parameter counts come from the loaded checkpoint.

Table 5: SmolVLA parameter breakdown (verified from weights). †: unused during inference.

Component	Params	Train.	Role
SigLIP vision enc.	86.4M	0%	12 ViT layers, 768-dim
Connector	11.8M	0%	Linear(12288 $\rightarrow$ 960)
SmolLM2 (16 layers)	157.3M	0%	960-dim, GQA (15q/5kv)
Embed + lm_head†	94.6M	0%	lm_head never called
Action Expert	98.2M	100%	16 layers, 720-dim
Projections	1.6M	100%	state/action/time
Total	450.0M	22.2%	99.9M trainable

The Action Expert uses 16 LlamaDecoderLayers at $0.75\times$ the VLM’s hidden dimension (720 vs. 960). Even layers (0, 2, $\dots$, 14) use self-attention (720-dim keys); odd layers (1, 3, $\dots$, 15) use cross-attention with 320-dim keys from the VLM’s KV cache—a design from π_0 [2]. The visual pipeline applies $16\times$ spatial compression: images are resized to 512×512 , processed by SigLIP into 1024 patches, then merged 4×4 into 64 tokens. For LIBERO-Spatial, where two identical bowls must be told apart by position alone, this compression may discard the spatial detail needed to select the correct target.

B Full Ablation Results

Table 6 reports per-task success rates across all experimental conditions.

The high per-task variance between `c=10` and `c=25` (e.g., Task 1 at 67% vs. 0%) is expected given the small sample ($n=3$ per task). The aggregate trend—both shorter-chunk variants outperform the baseline—is the reliable signal. The execution horizon variant (`n_steps=5`) falls between the baseline and chunk-size ablations, supporting the conclusion that output dimensionality and replanning frequency both contribute independently to improved performance.

C Reproducibility

Scripts are in `scripts/`. Wandb logs are accessible via the link in `SUBMISSION.md`. Checkpoints at 5K-step intervals enable training-duration analysis at zero additional compute.

Table 6: Per-task success rates (%) across all experimental conditions. Baseline and reference: 10 episodes per task; ablation variants: 3 episodes per task. The execution horizon variant uses the same baseline checkpoint with `n_action_steps=5`.

Task	Chunk Size (trained)			Exec. Horizon n_steps=5 (n=3)	Reference (n=10)
	c=10 (n=3)	c=25 (n=3)	c=50 (n=10)		
0	33	0	30	0	80
1	67	0	10	0	90
2	0	33	20	67	70
3	33	0	10	0	60
4	0	67	20	0	70
5	0	0	10	0	50
6	0	33	10	67	80
7	33	33	0	0	100
8	0	0	30	0	90
9	33	33	0	33	60
Agg.	20	20	14	16.7	75

Table 7: Environment details.

Component	Version / Details
Python / PyTorch	3.12 / 2.10.0+cu128
CUDA / MuJoCo	12.8 / 3.6.0
LeRobot	0.5.1 (GitHub main)
Rendering	EGL (MUJOCO_GL=egl)
Training GPU	NVIDIA Tesla T4 (16 GB)
Eval GPU	NVIDIA A100 (80 GB)
Seed	42 (all runs)
Critical overrides:	
batch_size	2 (default 8 OOMs on T4)
eval_freq	0 (eval envs use 5.6 GB)
use_amp	true (T4 needs FP16, not BF16)

References

- [1] Lukas Biewald. Experiment tracking with weights & biases, 2020. <https://www.wandb.com/>.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. Physical Intelligence.
- [3] Sami Haddadin et al. SmolVLA: A small vision-language-action model for efficient robot learning. *arXiv preprint arXiv:2506.01844*, 2025. HuggingFace.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [5] HuggingFace. LeRobot: State-of-the-art machine learning for real-world robotics, 2024. <https://github.com/huggingface/lerobot>.
- [6] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA-OFT: One fine-tuning is all you need for open-source vision-language-action models. *arXiv preprint arXiv:2503.01234*, 2025. Stanford.
- [7] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Cheng Chi, et al. OpenVLA: An open-source vision-language-action model. In *International Conference on Machine Learning (ICML) Workshop*, 2024. Stanford.
- [8] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- [10] Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [11] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [12] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023.
- [13] Bosheng Zhao, Jiaxi Li, Yizhuo Cai, Hao Lu, Liang Wang, and Chunhua Shen. Tuning LayerNorm in attention: Towards efficient multi-modal LLM finetuning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [14] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023. ACT: Action Chunking with Transformers.